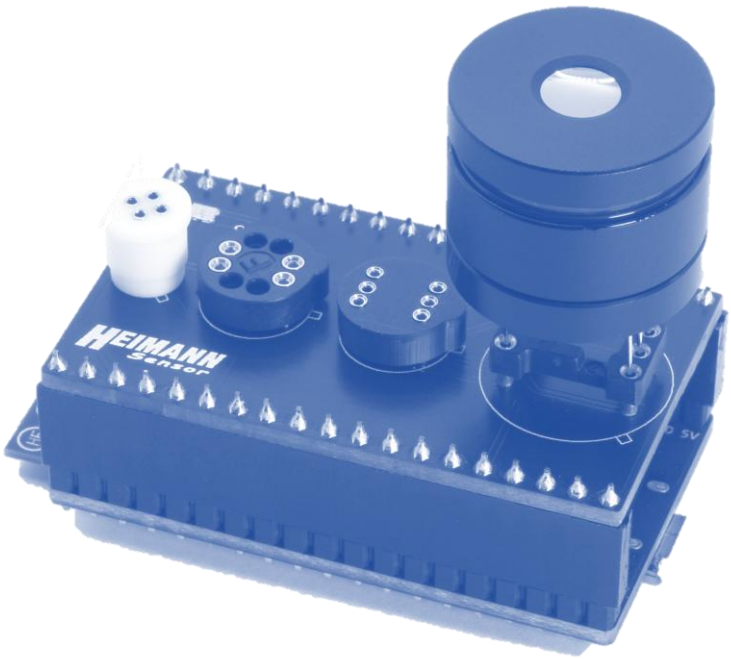
Authors: Christian Nägler; Paulina Bobinska
Revision 2
27.07.2026

USER MANUAL

Application Shield ESP32



DISCLAIMER

We reserve the right to correct or update the content, technical specifications or product descriptions at any time without prior notification.

All information is provided “as is”, without any warranty, express or implied, including but not limited to warranties of merchantability or fitness for a particular purpose.

To ensure you are using the most up-to-date and accurate version, please always refer to the latest edition available on our official website.

Only the version published on the website shall be considered authoritative and current.

HEIMANN Sensor GmbH
Maria-Reiche-Str. 1
D-01109 Dresden / Germany

Contact / Customer Support
Phone 49 (0) 6123 60 50 30
sales@heimannsensor.com

Internet
www.heimannsensor.com
info@heimannsensor.com

Content

1 Overview 4
2 Installation 7
3 Connect the Application Shield 10
4 Upload the Application Shield Firmware 10
5 Short Description of the Source Code 13
6 LED Status 15
7 Possible Errors 16
8 PCB Design 17
9 Schematic 18

1 Overview

This application shield is designed for use with an ESP32 development board. It provides easy access to our thermopile arrays, providing a fast and convenient way to acquire thermal images from the sensor. The complete source code is provided and is fully open source. It contains all required steps from reading the EEPROM to the calculation of the thermal image. The C++ code can be viewed and modified via the Arduino IDE. The PCB is designed as an ESP32-DevkitC-32D extension.

Supported Sensor types:

	TO-46	TO-39	TO-8
I²C	8x8d	16x16d 32x32d	/
SPI	/	60x40d	80x64d 84x60d 120x84d 160x120d

The source code includes three ways to interact with the sensor:

- via WiFi: thermal image streaming in the GUI
- via serial monitor: the sensor data as text output
- as Access Point: the ESP32 generates its own WiFi network and allows the connection to the PC

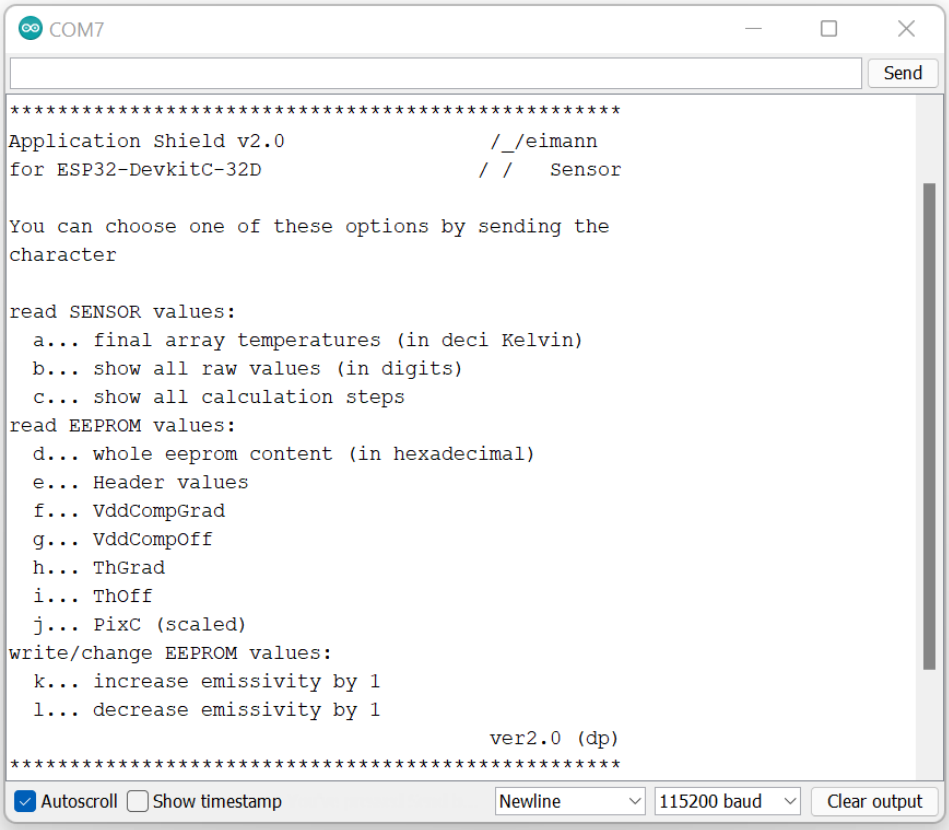
All modes are contained in the same code and can be activated by changing the matching define.

Serial

This mode prints all results in the serial monitor of the Arduino IDE. The EEPROM/Flash content and sensor voltages can be visualized here. It is easy to interact with the sensor by sending characters depending on the selected menu function.

Benefits:

- shows EEPROM/Flash content in hexadecimal or associated data type (float, short, long, ...)
- prints results after each calculation steps
- understands the process from raw pixel voltages to the final compensated thermal image

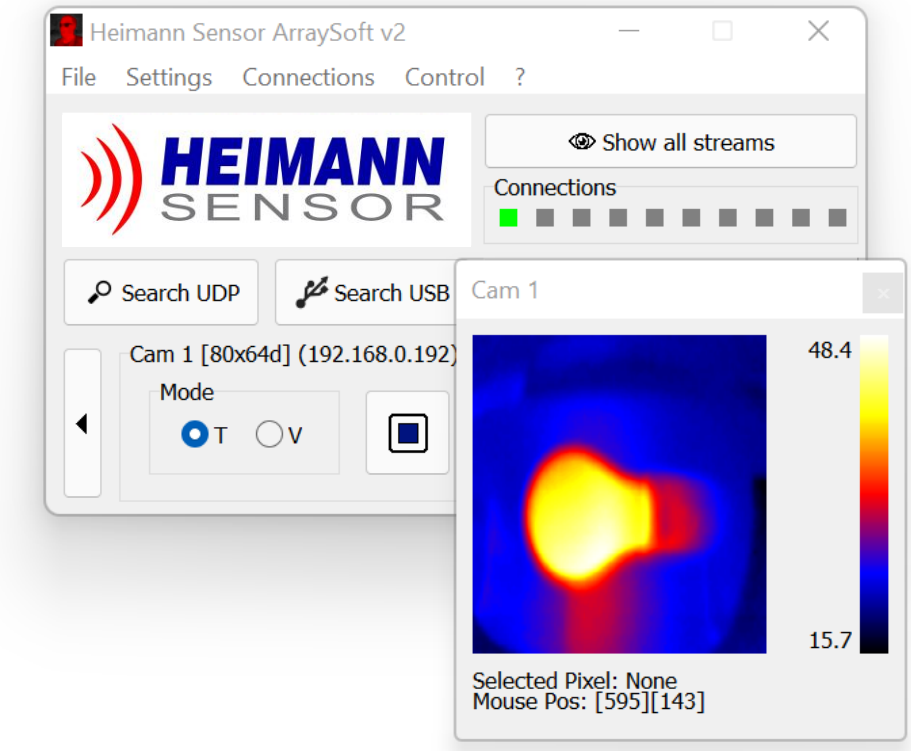


WiFi | AccessPoint

The thermopile can be connected to the Heimann Sensor GUI via WiFi to continuously stream. Using the GUI, sensor images can be streamed in either temperature or voltage mode. The GUI also allows the configuration of various user settings, such as the clock, ADC resolution, and emissivity factor.

Benefits:

- False-colour visualization of images
- stream continuously
- switch between temperature and voltage mode
- record/replay
- change user settings

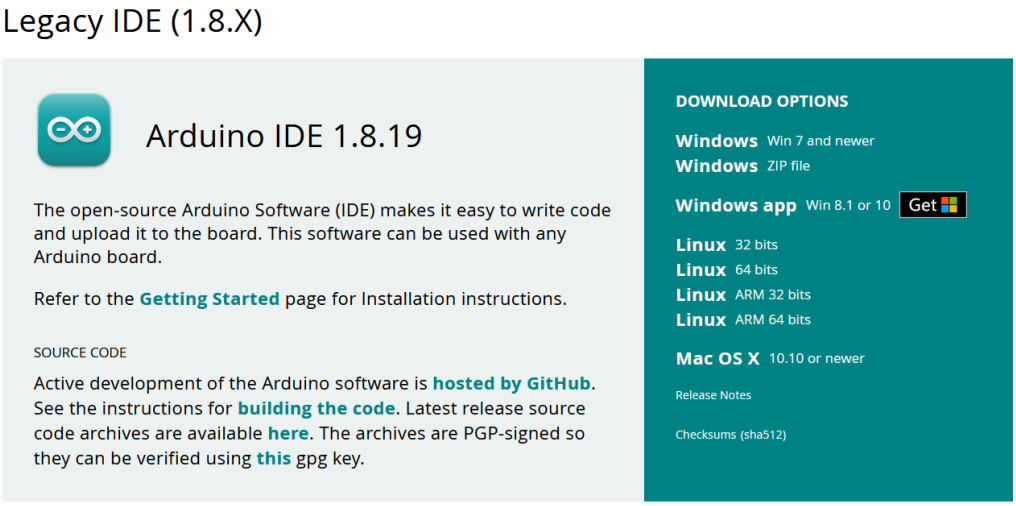


2 Installation

Follow these steps to start using the ESP32-DevkitC-32D with the Arduino IDE:

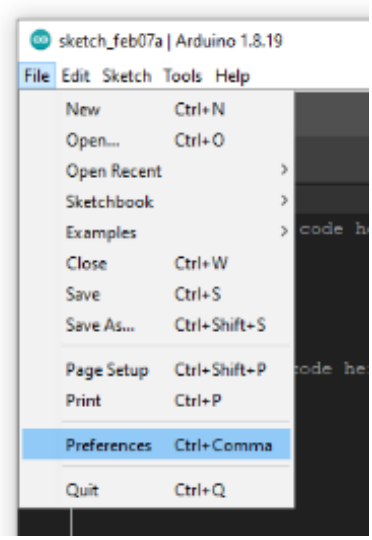
1. Download and install the Arduino IDE version **1.8.19** from:

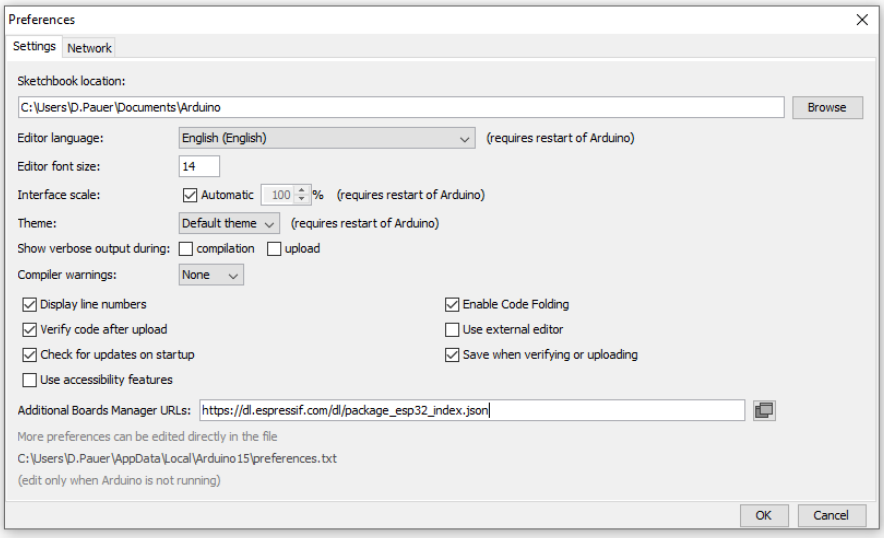
https://www.arduino.cc/en/Main/Software



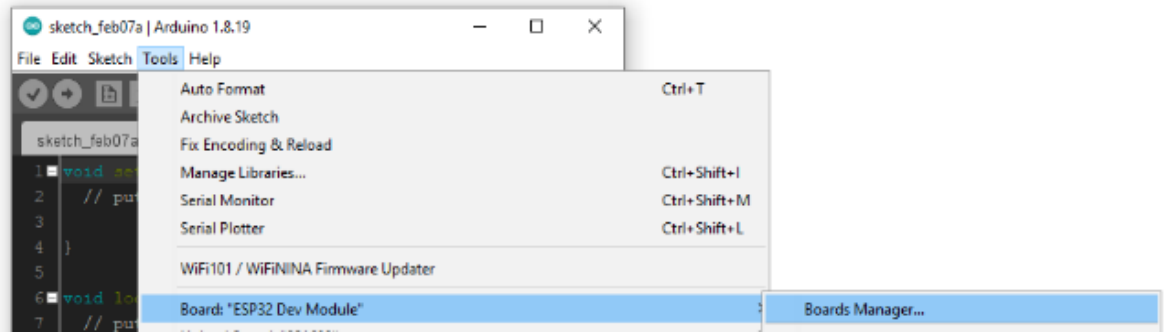
1. In **File > Preferences** you will find an input box called **Additional Boards Manager URLs:** Please copy the following link into this text box:

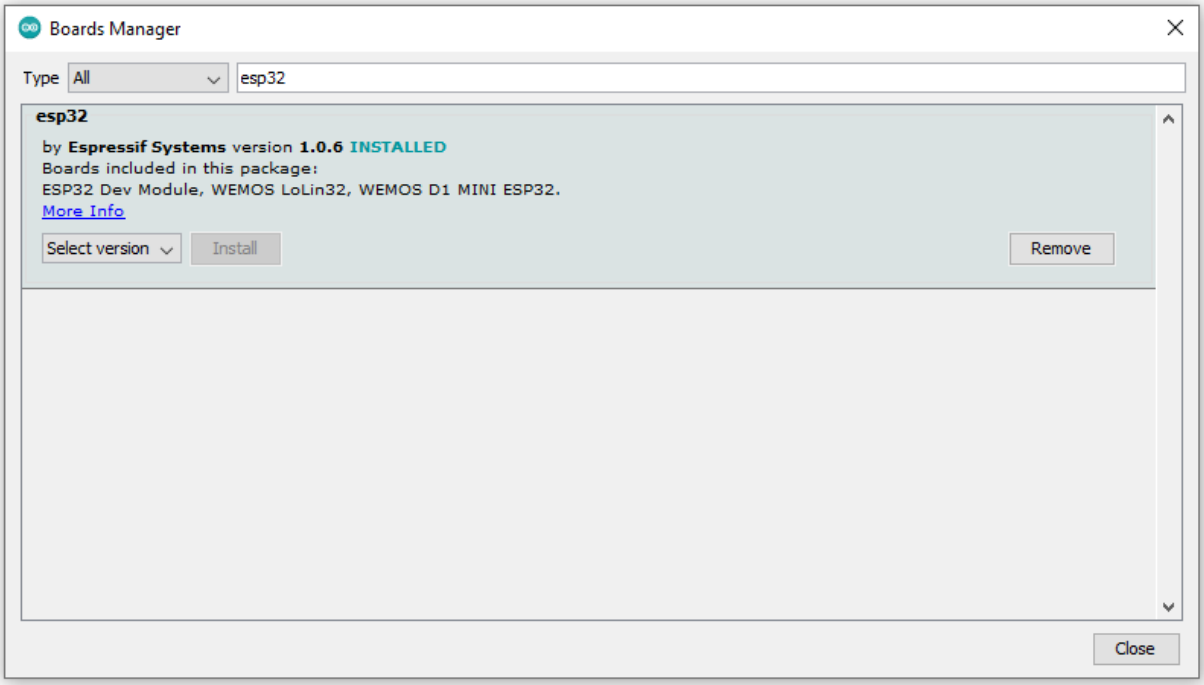
https://dl.espressif.com/dl/package_esp32_index.json





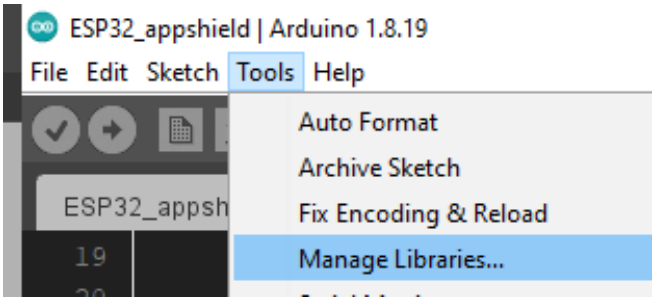
2. In **Tools > Board > Boards Manager** search for **esp32** and install the board library from **Espressif Systems**

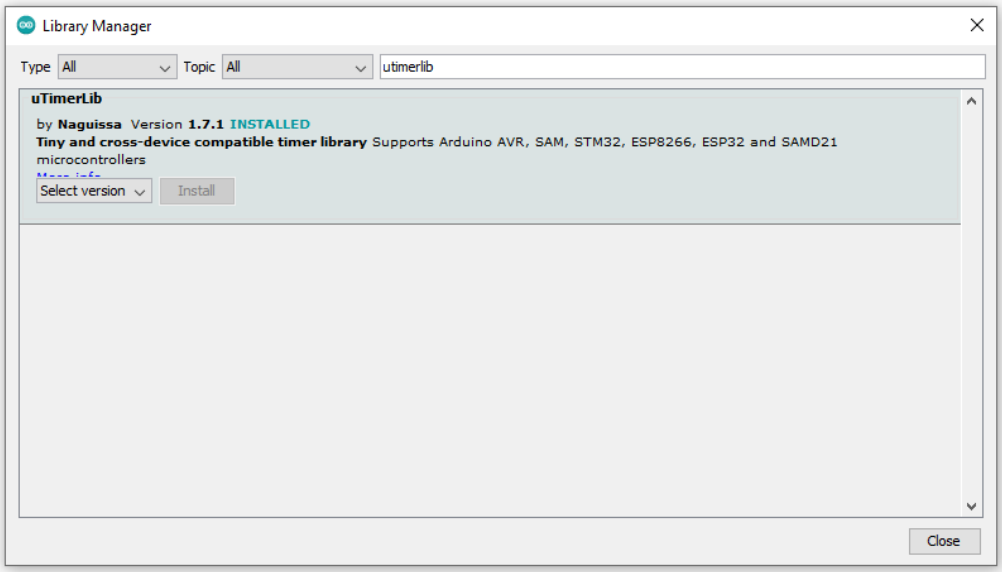




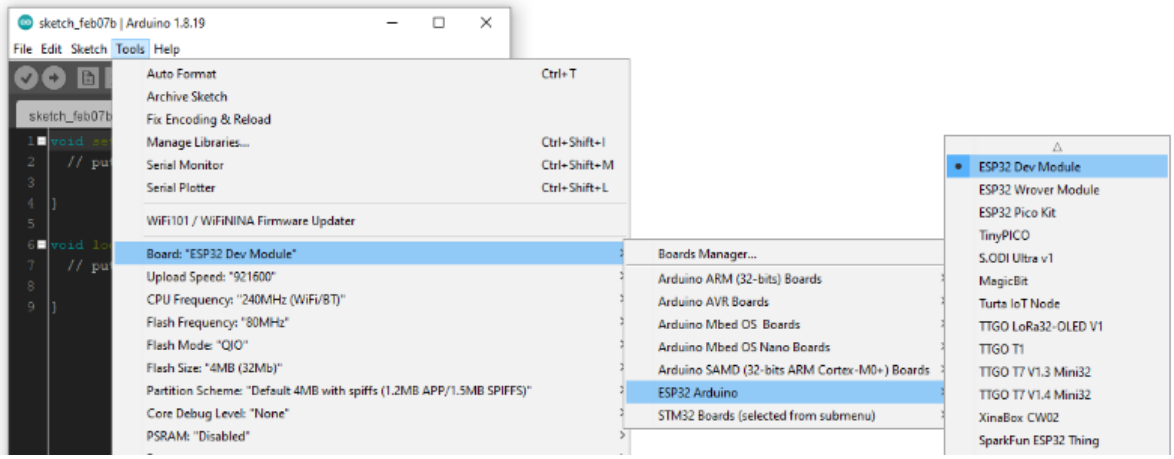
IMPORTANT: Please install version 1.0.6!
Until now there are some problems with newer versions.

3. Open the Library Manager in **Tools > Manage Libraries...** and search/install the library called **uTimerLib**



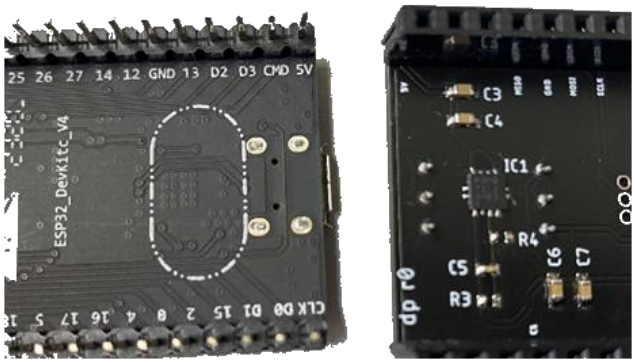


From now on, you can program the ESP32 board using the Arduino IDE. You will find it as **ESP Dev Module**. Do not change the default setting, such as Upload Speed, CPU Frequency, ...



3 Connect the Application Shield

To get the correct orientation between ESP32 board and Application Shield, have a look at the bottom side of both PCBs. There is a **5 V** label on each PCB.

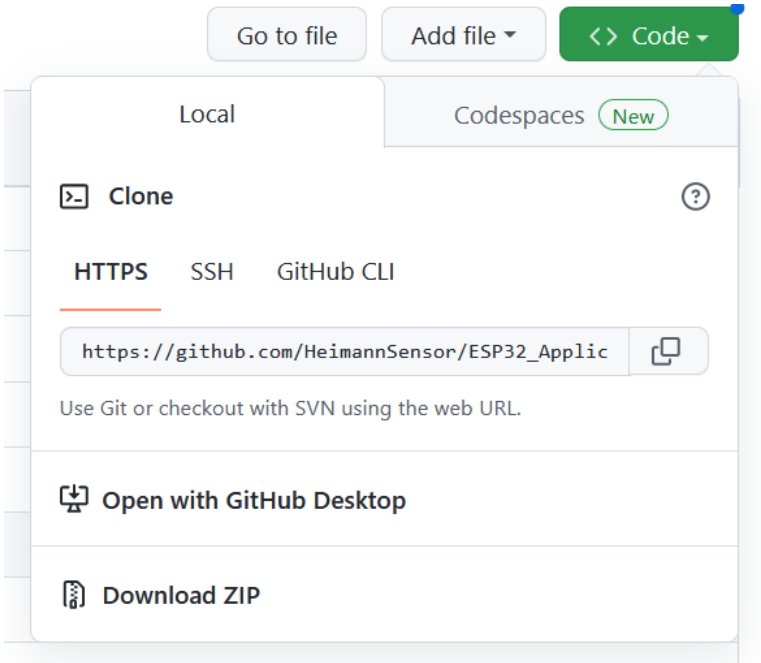


4 Upload the Application Shield Firmware

Please download the WiFi Shield software from GitHub:

https://github.com/HeimannSensor/ESP32_ApplicationShield

(click on <> **Code and Download ZIP**)



The program folder contains at least two files

- the **.ino** file includes the complete source code
- **def.h** includes all required settings and defines for each individual sensor. All def.h files are sorted in a folder structure called **all_sensor_defs**. You have to select the correct def.h for your sensor.
- **Wire.cpp** and **Wire.h** are the modified I²C library files. They only in program folders for 8x8, 16x16 and 32x32

Open the program by double-clicking the .ino file.

At the top of the source code you will find the two defines to enable/disable the **WiFi**, **ACCESSPOINT** and/or the **Serial** mode.

```
/*** MODES ********************************************************************************************
  The source code includes three ways to interact with the sensor:
  - via WIFI you can stream thermal images in our GUI (#define WIFIMODE)
  - via the SERIAL monitor you can observe the sensor data as text output (#define SERIALMODE)
  - via ACCESSPOINT the ESP32 creates the wifi network. You have to connect your computer to this network
    to stream thermal images in the GUI (#define ACCESSPOINT)
  All modes are contain in the same code and you can activate by activating the matching define.
*****************************************************************************************************/
//#define WIFIMODE
#define SERIALMODE
#define ACCESSPOINT // automatically diablead if WIFIMODE is active
```

Connect the ESP32 to Your WiFi Network

If you want to use the WiFi part, you have to change the variables **ssid** and **pass**. Please use the network name and password of your WiFi router.

```
/*****************************************************************************************************
  NETWORK INFORMATION
  - - - - - - - - -
  If you want to use the WIFI function, you have to change ssid and pass.
*****************************************************************************************************/
char ssid[] = "yourSSID";
char pass[] = "yourPassword";
```

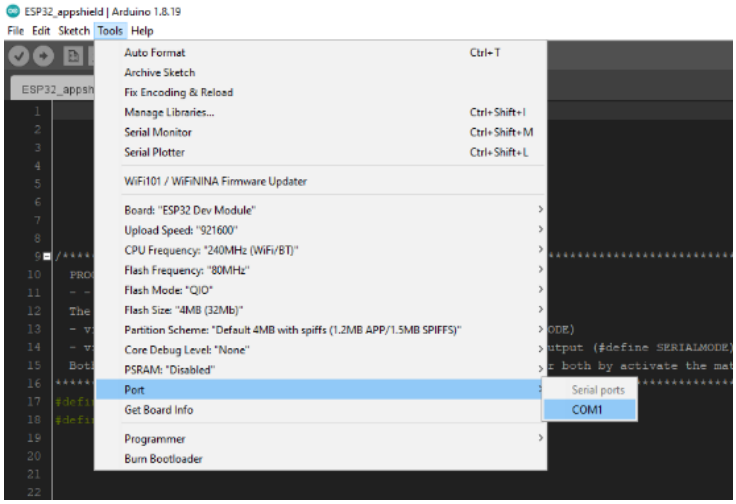
ESP32 Creates New WiFi Network

If you want to use the ESP32 as an access point, you will find the network name and password in the def.h file. Now you have to connect your device (Laptop, PC, ...) to the ESP32 network to find the sensor in our GUI.

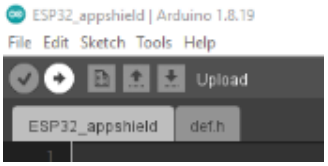
```
#define ACCESSPOINTNAME "HTPAd8x8L2k1"
#define ACCESSPOINTKEY "heimannsensor"
```



Select the matching COM number...



... and upload the program.



If the upload is done, you are able to interact with the sensor with the chosen communication mode (serial and/or WiFi).

Serial Communication

Open the serial monitor and send the character “m” to see the menu. The menu will show you what options are possible with the serial monitor.

WiFi and/or Access Point Communication

To connect the ESP32 with the GUIv2: Open the GUI and press on “**Search UDP**”. Now you find a PLAY button to start the stream. For more information: please have a look into the user manual of our GUI “Heimann Sensor ArraySoft v2”.

Possible Errors During Uploading

If you get this error message:

```
Connecting........____.....____.....____.....____.....____.....____.....___An error occurred while uploading the sketch
_
A fatal error occurred: Failed to connect to ESP32: Timed out waiting for packet header
```

You only have to press the Boot-button on the ESP32-board during this ....____....__ part.

5 Short Description of the Source Code

The program checks first if there is a sensor connected to the board and if the WiFi connection is available. If both are true, the ESP32 reads the complete EEPROM/Flash and starts polling sensor data without any output. The sensor will permanently operate in the background to reach a state of thermal stability. A software timer interrupts the other functions if there are new sensor data available. There are two bigger arrays to handle all pixel data. One saves new raw pixel data from the single blocks in 8-bit format and the second array (nxm size; depends on the chosen sensor) contains the sorted pixel data in 16bit format. The second one will be used for the pixel data compensation as explained in the datasheet. If there is a request for an output (via GUI and/or serial monitor) the ESP32 will handle that after the next fully compensated image.

Besides the declaration part at the top of the source code, the complete code can be divided in four parts for better understanding:

Part 1: Arduino function

This part include the both Arduino specific functions: setup() and loop(). Both are required for each Arduino code. The setup() function is only called once at the beginning and the loop() works as the main-loop of this code. Also in part 1 are functions to handle the LED on the board and the interrupt service routine.

Part 2: HTPAd functions

This part contains all functions for handling the data from thermopile array. Here you will find:

- the routine to read the whole EEPROM/Flash
- the read routine to get new raw pixel voltages from the sensor
- the sort routine to convert the 8bit block data to the 16bit nxm array (size depends on the array size)
- all calculation steps to get the final pixel temperatures in dK (deci Kelvin)
- pixel masking routine (if the sensor has defective pixels)
- write examples to write in the EEPROM (not for devices with flash)

Part 3: WiFi functions

In this part the WiFi will be initialized. The code checks incoming UDP packets and communicates with our GUIv2.

Part 4: Serial functions

This part handles the whole serial communication between ESP32 and Serial Monitor of the Arduino IDE. This part contains functions to print the menu, all EEPROM/Flash values and the sensor data in m×n format.

6 LED Status

There is a three-colour LED in the upper left corner on the PCB. The following table lists the meaning for each LED state:

LED STATE	Description
BLUE (constant)	ESP32 is trying to connect with the WiFi network. If this state does not change, please check your WiFi user settings in the code and/or the WIFI setup in your environment. A normal duration for this state is 10 – 60 seconds.
BLUE (blinking)	ESP32 is connected to the WiFi network. Now you can chat with the ESP32 via GUIv2 and/or Serial Monitor.
BLUE and GREEN blinking alternately	ESP32 works as Access Point and is ready for a connection. Now you can chat with the ESP32 via GUIv2 and/or Serial Monitor.
Light BLUE (blinking)	The GUIv2 called for a UDP device and bound this ESP32. The ESP32 only communicates to this GUI from now. You cannot find this device with other GUIs yet. Closing the GUI will release the device.
GREEN (blinking)	ESP32 sends the thermal images via UDP packets to the GUI and the WiFi signal strength is very good. This is the perfect state to stream continuously.
RED (blinking)	ESP32 sends the thermal images via UDP packets to the GUI and the WiFi signal strength is very poor. It is not guaranteed that the UDP packets will be received correctly by the GUI. Please have a look on your WiFi network and increase the signal strength (for example by using a shorter distance to the router).

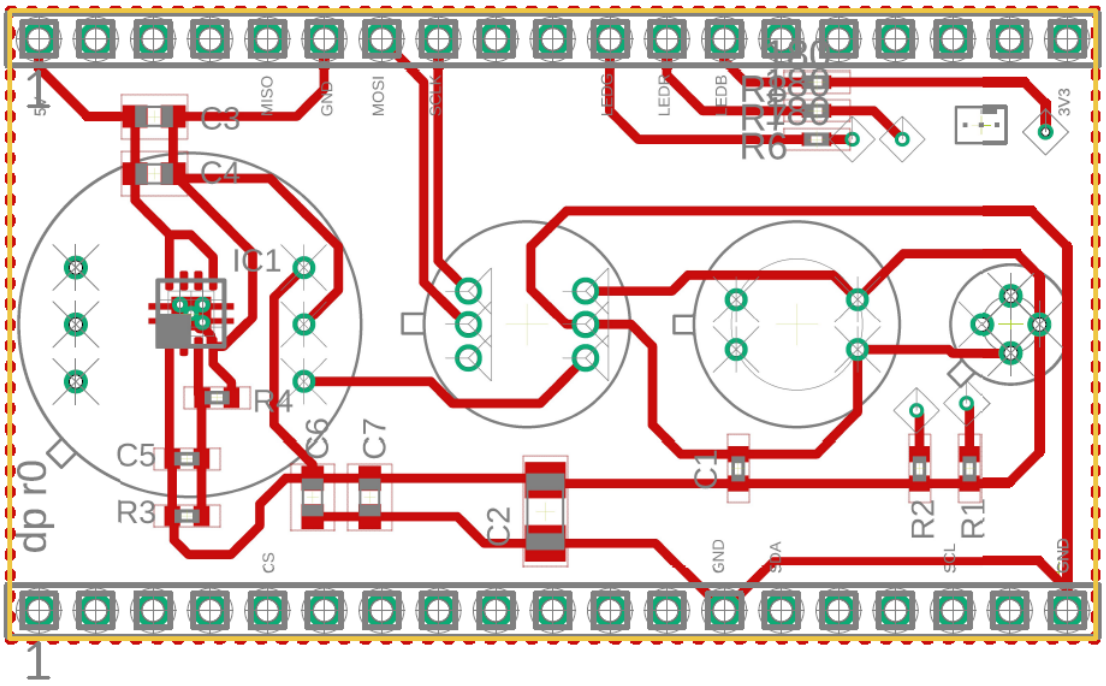
7 Possible Errors

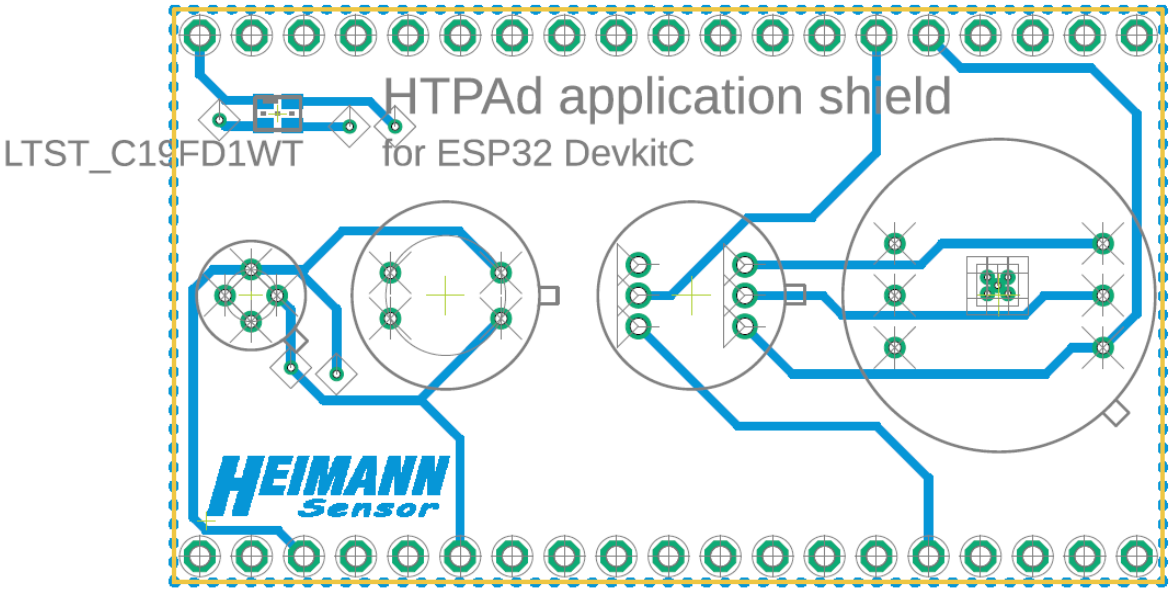
Wrong Look-up Table

If you see a little LUT sign in the right upper corner of the thermal image, you have included the wrong def.h file for your sensor. Please choose the correct def.h for your sensor and replace it with the current one in you program folder.

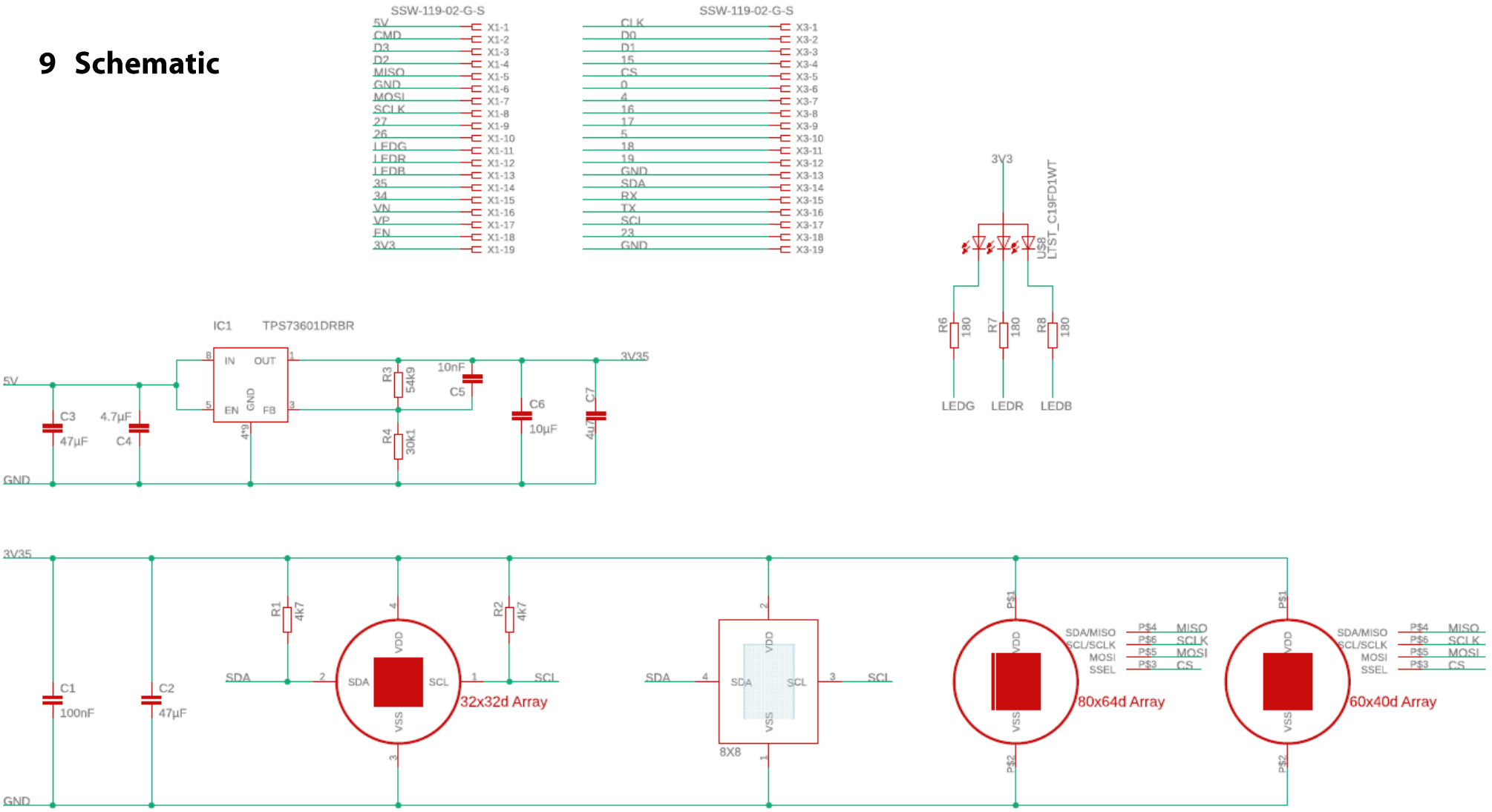


8 PCB Design


C3
C4
IC1
R4
C5
R3
C6
C7
C2
C1
R2
R1
MISO
MOSI
3V3
GND
CS
SDA
SCL


HTPAd application shield
LTST_C19FD1WT
for ESP32 DevkitC
HEIMANN Sensor

9 Schematic


SSW-119-02-G-S
5V
CMD
D3
D2
MISO
GND
MOSI
SCLK
27
26
LEDG
LEDR
LEDB
35
34
VN
VP
EN
3V3
X1-1
X1-2
X1-3
X1-4
X1-5
X1-6
X1-7
X1-8
X1-9
X1-10
X1-11
X1-12
X1-13
X1-14
X1-15
X1-16
X1-17
X1-18
X1-19
SSW-119-02-G-S
CLK
D0
D1
15
CS
0
4
16
17
5
18
19
GND
SDA
RX
TX
SCL
23
GND
X3-1
X3-2
X3-3
X3-4
X3-5
X3-6
X3-7
X3-8
X3-9
X3-10
X3-11
X3-12
X3-13
X3-14
X3-15
X3-16
X3-17
X3-18
X3-19
3V3
US8
LTST_C19FD1WT
R6
180
R7
180
R8
180
LEDG
LEDR
LEDB
IC1
TPS73601DRBR
IN
OUT
EN
GND
FB
5V
C3
47µF
4.7µF
C4
GND
R3
54k9
10nF
C5
R4
30k1
C6
10µF
C7
4u7
3V35
3V35
C1
100nF
C2
47µF
GND
R1
4k7
R2
4k7
SDA
SCL
VDD
VSS
32x32d Array
8X8
P$1
P$2
SDA/MISO
SCL/SCLK
MOSI
SSEL
P$4
P$6
P$5
P$3
MISO
SCLK
MOSI
CS
80x64d Array
60x40d Array